

Scalable Real-Time E-Commerce Clickstream Analytics

SHARMILA RAMARAJ | X24244066

MSc Cloud Computing, National College of Ireland | Individual submission | August 2026

Abstract- Online shops need to understand what customers are doing now without losing the context provided by earlier behaviour. I built a small, reproducible AWS Lambda architecture that replays a public REES46 electronics clickstream into Amazon Kinesis. AWS Lambda processes the recent events, DynamoDB stores minute-level counters, and PySpark on Amazon EMR calculates historical five-minute baselines. A Python serving layer and an AWS-hosted dashboard combine the two views to identify unusual product activity and show where sessions drop out of the purchase funnel. Of 200,000 selected source rows, 199,965 were valid. Five S3 copies formed a 999,825-event benchmark of approximately 290 MB. On the same two-worker EMR cluster, the one-partition reference completed in 70 s and the eight-partition run completed in 58 s, giving a 1.207x speedup and a 17.1% reduction in duration. The improvement is useful but modest, and the report explains the Spark start-up, storage and coordination costs that limited it.

Keywords- clickstream analytics, Lambda architecture, Amazon Kinesis, AWS Lambda, DynamoDB, PySpark, Amazon EMR, auto-scaling

I. INTRODUCTION

Online stores generate product views, cart actions, removals and purchases continuously. Historical reports can show long-term popularity but may hide a sudden spike, while a live counter can mistake normal popularity for an anomaly. The practical question addressed here is: Which products have unusually high activity during the latest five-minute window, and is the larger current funnel drop-off view-to-cart or cart-to-purchase?

I chose this question because a raw list of popular products is not enough. A product may always be popular, so the current score must be compared with its own historical level. The resulting view can support promotion, stock checks and investigation of purchase friction. It does not identify customers, change prices or contact users. The contribution is an end-to-end Python implementation with reproducible input, separate batch and speed paths, an explicit serving merge, and measured parallel execution on AWS Academy infrastructure.

A. Objectives

The objectives were to: (1) build a controlled replay producer; (2) compute historical baselines with distributed PySpark; (3) calculate recent product and funnel metrics from Kinesis; (4) combine current and historical views; (5) configure elastic AWS processing; and (6) compare sequential-reference and parallel execution using the same input and cluster.

II. BACKGROUND AND DESIGN RATIONALE

The Lambda architecture separates a complete, recomputable batch view from a low-latency speed view and merges them for queries [1]. This separation fits the use case: batch computation supplies accurate historical norms, whereas recent counters reveal changes early. Spark provides distributed, fault-tolerant transformations derived from the resilient distributed dataset model [2]. Kinesis and Lambda provide a managed event path in which Lambda polls the stream and invokes the consumer for available records [3].

A single stream-only design would be simpler but would require durable state, replay and historical recomputation to answer the same question. A batch-only design would be accurate but stale. The selected hybrid therefore prioritises both correctness and freshness while remaining small enough for a Learner Lab.

III. USE CASE AND DATA

A. Operational Metrics

For product p in the latest five minutes, recent engagement is weighted so that actions nearer to conversion contribute more:

$$\text{trend_score}(p) = \text{views}(p) + 3 \text{ carts}(p) + 5 \text{ purchases}(p).$$

The batch layer computes the historical mean of the same score over observed five-minute windows. Activity lift is the current score divided by this historical mean; a product is labelled unusually trending when lift is at least 2.0. This rule detects relative change, so a normally popular product is not automatically treated as anomalous.

$$\text{view-to-cart drop-off} = 1 - \text{cart sessions} / \text{view sessions}$$

$$\text{cart-to-purchase drop-off} = 1 - \text{purchase sessions} / \text{cart sessions}.$$

These ratios are window indicators, not proof that a named customer permanently abandoned checkout. A separate cart signal is generated after 15 replay-clock minutes if a session has neither purchased nor removed that product.

B. Dataset Preparation

The REES46 source contains anonymised e-commerce behaviour from an electronics and home-appliance store collected between October 2019 and February 2020 [4]. Its fields include event time and type, product/category, brand, price, user and session identifiers. The source contained 885,129 events. I selected the first 200,000 rows, retained 199,965 valid events, and rejected 35 rows (0.0175%) without a session identifier. This was a deliberate quality rule: without a session ID, a view, cart or purchase cannot be placed reliably in the same funnel journey.

Item	Verified value
Source events	885,129
Selected / valid	200,000 / 199,965
Rejected	35 (0.0175%)
Canonical JSONL	58 MB
Benchmark	999,825 events; ~290 MB

IV. SYSTEM ARCHITECTURE

Implemented Lambda Architecture

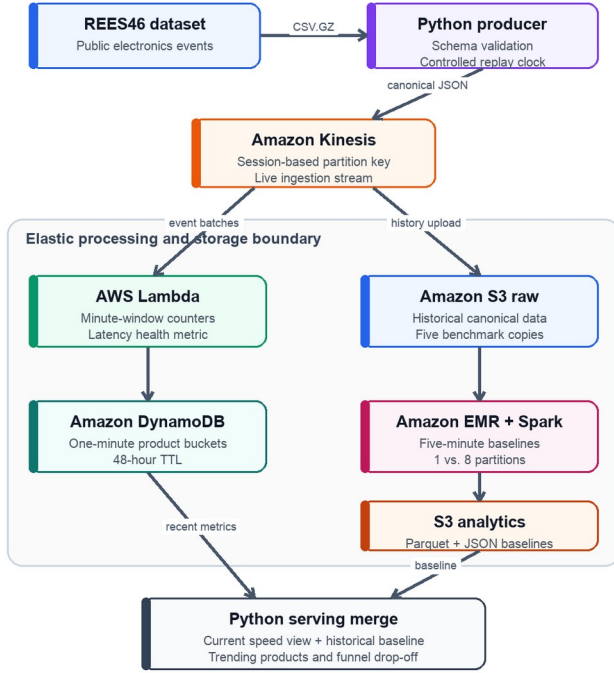


Fig. 1. Implemented batch, speed and serving paths.

The producer normalises the historical CSV.GZ file into canonical JSON, preserves the source timestamp and assigns a replay-clock event time. Session ID is the Kinesis partition key, preserving per-session order within a shard. The live branch invokes a Python Lambda and uses atomic DynamoDB updates for product and health counters in one-minute buckets with time-to-live expiry. The historical branch stores canonical input in S3, where EMR Spark aggregates five-minute windows and writes Parquet plus consolidated JSON baselines.

The serving module reads the recent view and historical product rows, calculates lift, orders products by relative change, and returns a JSON analytical view. A responsive static dashboard on an Amazon S3 website presents the same verified values as product activity, a three-stage funnel, service status and batch-performance panels. CloudWatch and retained EMR logs provide operational evidence.

A. Tools and AWS Services

Component	Role in the project
Python / boto3	Replay, validation, Lambda and serving merge
Kinesis	Session-partitioned live ingestion
Lambda	Low-latency incremental counting
DynamoDB	One-minute speed-layer buckets
S3	Raw history, baselines and static dashboard
EMR / PySpark	Distributed historical aggregation

V. IMPLEMENTATION

A. Reproducible Ingestion

The Python producer accepts CSV or compressed CSV, maps source event types to the canonical schema, validates timezone-

aware timestamps and required identifiers, and supports a record limit, target replay rate and no-sleep benchmark mode. The 199,965-event normalisation completed in 16.804 s, equivalent to approximately 11,900 records/s for local transformation and file output. This figure is not presented as end-to-end Kinesis throughput.

B. Batch Layer

The PySpark job reads JSON from S3, filters invalid rows, groups product events into five-minute windows, pivots event types and calculates activity scores. A second aggregation produces mean views, carts, purchases and trend score per product, while distinct session counts provide historical funnel context. Outputs are written as compressed Parquet for efficient analytical access and JSON for the serving demonstration.

C. Speed and Serving Layers

The Kinesis event-source mapping invokes the Lambda in batches. For each supported event the function increments the corresponding DynamoDB attribute and a health row containing event count and accumulated processing latency. Partial batch failure identifiers are returned so failed Kinesis records can be retried. The deterministic eight-event fixture produced three rows: two product aggregates and one health row.

The local stateful implementation maintains an exact five-minute deque and session counters for repeatable tests; the AWS Lambda persists equivalent one-minute buckets. The speed-layer screen keeps the current counts and funnel together, which made it easier for me to check that the same four view sessions, three cart sessions and one purchase session were carried into the visual result.

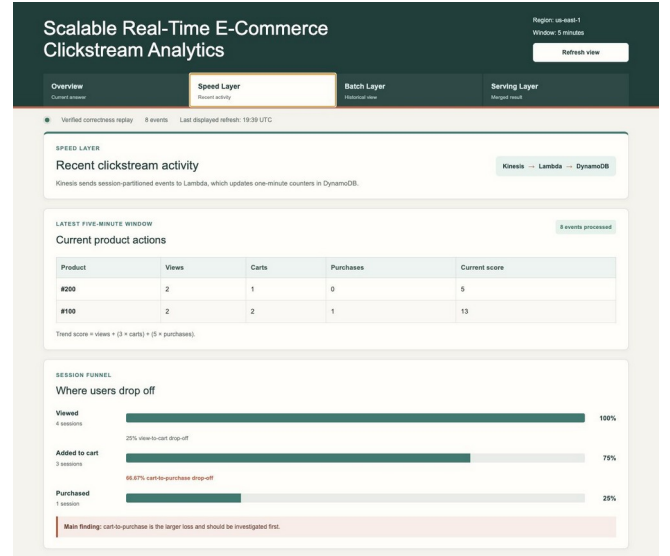


Fig. 2. Speed-layer view showing current actions and funnel loss.

D. Elasticity and Fault Handling

The EMR 7.13.0 cluster used Spark 3.5.6, one primary node, one core worker and one task worker at launch. Managed scaling was configured for a minimum of two and maximum of four worker instances, with at most one core worker. AWS documents that managed scaling evaluates cluster workload and can increase or decrease core/task capacity [5]. Lambda concurrency can expand with incoming event batches within account and event-source limits. The cluster also used a 20-minute idle-termination control and was manually terminated after measurement.

VI. EXPERIMENTAL METHOD

I tested correctness before attempting a larger benchmark. Eight deterministic events covered two products and four sessions. After the event-source mapping reached Enabled, the replay populated DynamoDB and I compared the serving JSON with the expected funnel and trend values. Eight automated Python tests covered the models, producer behaviour, speed logic, sequential baseline and serving merge; two dashboard checks verified the exported interface.

For batch performance, five S3-side copies of the 199,965-event canonical subset formed one 999,825-event input of approximately 290 MB. Both runs used the same EMR cluster and program. The reference forced one Spark partition; the parallel case repartitioned by product into eight partitions. Duration was taken from completed EMR step timings. Speedup was calculated as $T(1 \text{ partition}) / T(\text{configuration})$. This controls the data and infrastructure but varies partitions rather than worker count.

VII. RESULTS

A. Real-Time Analytical Answer

In the controlled serving result, product 200 scored 5 against a historical five-minute mean of 2.5. Its activity lift was 2.0, so it was flagged as unusually trending. Product 100 scored 13 against a mean of 13; lift was 1.0 and it was not flagged. Across the four view sessions, three reached cart and one reached purchase. Therefore view-to-cart drop-off was 25%, while cart-to-purchase drop-off was 66.67%. The larger immediate loss was after cart, suggesting that product/checkout friction should be investigated before adding more top-of-funnel promotion.

B. Serving Dashboard

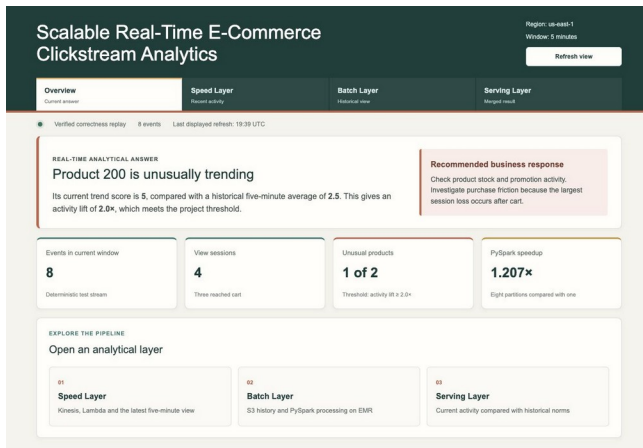


Fig. 3. Dashboard overview and clickable analytical layers.

The dashboard opens with the answer instead of the architecture. Product 200 is shown at 2.0x activity lift, followed by the four headline measurements. The Speed, Batch and Serving buttons open focused views of the same pipeline. This keeps the interface useful as an analytics application while the full AWS architecture remains available in Fig. 1 and in the implementation discussion.

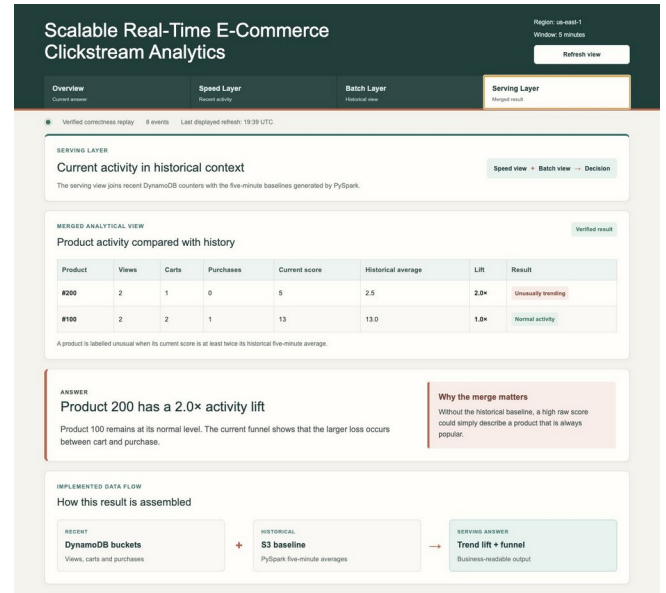


Fig. 4. Serving view combining recent and historical product activity.

The serving tab makes the Lambda merge visible without asking the reader to interpret raw AWS records. Product 200 has a current score of 5 against a baseline of 2.5, while product 100 remains at its normal level. The short flow at the bottom shows that DynamoDB supplies the recent view and S3 supplies the PySpark baseline.

C. Batch Performance

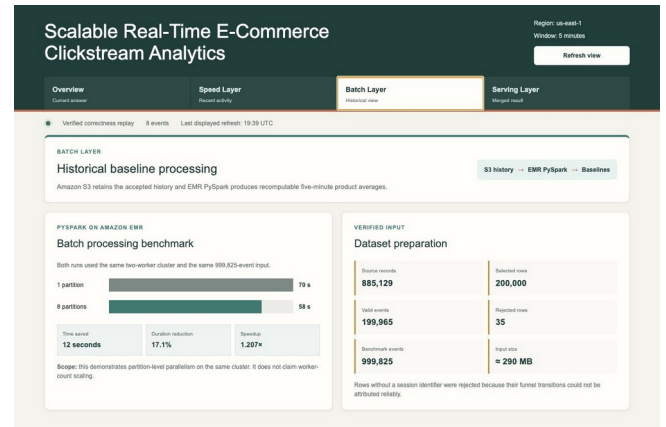


Fig. 5. Batch tab with verified input and EMR timing.

PySpark Batch Benchmark (999,825 Events)

Same EMR cluster | two initial workers | 290 MB input



Fig. 6. Same-cluster partitioning comparison on EMR.

Configuration	Part.	Time	Speedup
Sequential reference	1	70 s	1.000x
Parallel execution	8	58 s	1.207x

The parallel run completed 12 s faster, reducing elapsed duration by 17.1%. Output verification found two Parquet part files, one consolidated JSON file, success markers and approximately 9.75 MB of results. The 1.207x gain is useful but well below linear because Spark startup, S3 I/O, JSON parsing, shuffle coordination, task scheduling and final coalescing remain fixed or partly serial. The modest input also makes distributed overhead a larger fraction of total time.

VIII. CRITICAL ANALYSIS

The part I consider strongest is reproducibility. The source is public, the small fixture has known answers, the formulas are explicit, and both Spark runs used the same input. This made it possible to explain why product 200 was flagged instead of presenting a dashboard value as a black box. The Lambda architecture also earns its complexity here: the speed layer supplies freshness, but only the batch baseline tells us whether the current score is genuinely unusual.

The experiment also has limits that should not be hidden. One versus eight partitions on the same two-worker cluster is a parallelism comparison, not a controlled node-scaling experiment, so I do not claim worker efficiency. The managed-scaling policy was configured, but the 58-70 s jobs were too short to create a sustained backlog and prove a scale-out event. The cloud correctness replay contained eight events; the larger dataset was used for preprocessing and EMR batch execution, not for a sustained Kinesis load test. Finally, DynamoDB atomic increments are not idempotent, so a successfully processed record that is retried could be counted twice.

Production improvements would add an event ID and deduplication record, emit percentile latency rather than only accumulated latency, retain raw stream records through a managed delivery path, and run repeated trials across several worker counts and data sizes. A longer backlog-generating load test should capture actual scaling timestamps, Kinesis iterator age, Lambda errors/throttles, executor utilisation and cost. Structured Streaming is also a viable future consolidation path because it supports event-time windows, checkpointing and incremental execution [6], although it would change the deliberately separate Lambda speed layer used here.

IX. CONCLUSION

The project answered its main question. Product 200 was unusually active relative to its own history, and the larger session loss occurred after cart. Kinesis, Lambda and DynamoDB supplied the recent view; S3 and EMR supplied recomputable history; and the serving merge turned both into a readable decision. The EMR comparison measured a 1.207x speedup from eight partitions, while the report avoids presenting that result as proof of worker scaling. If I continued the project, I would first run longer repeated load tests, add idempotent event handling, and capture an observed EMR scale-out rather than only the configured policy.

REFERENCES

- [1] N. Marz and J. Warren, *Big Data: Principles and Best Practices of Scalable Real-Time Data Systems*. Manning, 2015.
 - [2] M. Zaharia et al., 'Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing,' *Proc. NSDI*, pp. 15-28, 2012. usenix.org/conference/nsdi12/technical-sessions/presentation/zaharia
 - [3] Amazon Web Services, 'Using AWS Lambda with Amazon Kinesis,' *AWS Lambda Developer Guide*, n.d. [Online]. Available: docs.aws.amazon.com/lambda/latest/dg/with-kinesis.html.
 - [4] REES46, 'Events in an electronics and home-appliance store,' dataset covering Oct. 2019-Feb. 2020. [Online]. Available: data.rees46.com.
 - [5] A. Sinha, J. Marques, S. Addanki, and V. Vyas, 'Introducing Amazon EMR Managed Scaling,' *AWS Big Data Blog*, Jul. 7, 2020.
 - [6] Apache Software Foundation, 'Structured Streaming Programming Guide, Spark 3.5.6,' May 2025. [Online]. Available: spark.apache.org/docs/3.5.6/structured-streaming-programming-guide.html.
- Submission links-** Repository: github.com/Sharmila-Ramaraj/scalable-clickstream-analytics
AWS dashboard: <http://scalable-real-time-clickstream-analytics-x24244066.s3-website-us-east-1.amazonaws.com> | Video: [\[INSERT YOUTUBE OR ONEDRIVE URL\]](#)